連珠AI
ソースコード仕様書

作成日 2026年10月4日

このプログラムは何をするものか

連珠の盤面とAIの石の色を受け取り、次に置く石の座標を1つ返すC言語のAI部品です。今すぐ勝てる手や必要な防御を優先し、その後は相手の応手を先読みして候補を比較します。

項目	概要
公開する関数	calcPutPos() の1つ
入力と出力	入力: 盤面・AIの石色 / 出力: x座標・y座標・成功/失敗
基本方式	単一スレッドのαβ探索 + 反復深化 + NNUE評価
既定の盤面	NNUE利用時は15×15。空盤の初手は中央 (7, 7)。
既定の規則	黒の三三・長連を禁止。黒の四四は禁止しない。
モデルとデータ	形状表と黒/白の学習重みをソースに埋め込む。

AIと対局プログラムの役割分担

AIは「どこに置くか」を決めます。画面表示、ユーザー入力、実際の石の配置、勝敗確定、手番の管理は呼び出す対局プログラムが行います。成功時も、入力盤面にAIが石を置くことはありません。

読み方

読む目的	該当章
使い方と対局の挙動を知る	02 入出力 / 03 処理順 / 04 規則 / 05 候補
先読みの仕組みを理解する	06 探索 / 07 評価 / 08 内部状態
組み込み・設定・保守を行う	09 設定 / 10 資源・ビルド / 11 失敗と制約 / 12 関数・確認項目
設定値を正確に参照する	付録A: 変更可能な設定119項目

本書は添付ソースの静的解析に基づく仕様です。gomoku.h・対局側の実装は未添付のため、その内容や実機での性能・勝率は検証していません。コメントの意図と実処理が異なる場合は、実処理を記載しています。

02 公開関数・入出力

呼び出し側は、有効な盤面と別々の座標出力先を用意して呼び出します。

```
int calcPutPos(int board[][BOARD_SIZE],
int computer_stone,
int *output_x, int *output_y);
```

引数	方向	仕様
board	入力	board[y][x] で石を参照。BOARD_SIZE 行×列の有効な領域を渡す。
computer_stone	入力	STONE_BLACK または STONE_WHITE。AIが置く石の色。
output_x	出力	横方向の座標を書き込む有効な int*。
output_y	出力	縦方向の座標を書き込む有効な int*。output_x と別アドレス。

座標と石の表現

座標は0から BOARD_SIZE-1 までです。x は配列の第2添字、y は第1添字です。15×15なら (0,0) は board[0][0]、(7,7) は中央の board[7][7] を指します。画面の上下方向の表示は対局側の仕様に従います。

内部処理は空点=0、黒=1、白=2を前提にしています。gomoku.h の STONE_SPACE、STONE_BLACK、STONE_WHITE は、この数値に一致させる必要があります。

戻り値	出力の意味
0: 成功	合法な候補から選んだ (x,y) を返す。盤面に石は置かない。
1: 失敗	入力不正、初期化失敗、領域確保失敗、満盤、候補なしなど。書込み可能な出力先は入口で -1 に初期化する。

呼び出し例

```
int x, y;
int status = calcPutPos(board, STONE_WHITE, &x, &y);
if (status == 0) {
board[y][x] = STONE_WHITE;
/* Continue game handling here. */
}
```

出力先は盤面や他の入力領域と重ねないでください。出力を -1 に初期化してから引数を検査するため、不正なポインタを渡すこと自体は防げません。出力先が同一アドレスなら失敗を返します。

03 一手を決める処理順

公開関数の開始から終了まで。空盤と即勝ちは先読みを省略します。

順	処理	結果・分岐
1	出力を初期化し、入力を検査	NULL、出力先の重複、石色、盤面値を確認。異常なら失敗。
2	探索表とNNUEを初期化	初回に形状符号、局面キー、係数表、黒/白モデルを準備。
3	AI用の領域を確保	Position と探索スタックを確保。失敗時は解放して失敗。
4	外部盤面を読み込む	対局履歴を更新し、内部盤面を空から着手列で再構成。
5	満盤・空盤を確認	満盤なら失敗。空盤なら中央を返して成功。
6	根の候補を生成	序盤範囲の拡張、脅威に応じた候補生成、合法手選別、対称手除去。
7	候補と評価状態を準備	候補ごとの評価・主変化を初期化。初期静的評価を取得。
8	探索時計を開始	訪問数、中断フラグ、完了深さ、置換表の世代を更新。
9	即勝ち確認・反復深化	先頭候補が五連なら必勝値を設定。それ以外は深さ2から反復。
10	着手座標を返す	現在の候補配列の先頭を座標に変換。予想盤面/履歴を更新。
11	一手用の領域を解放	Position と探索スタックを解放し、成功0を返す。

繰り返し呼び出したとき

AIは、前回返した手まで置かれたと予想した盤面を保持します。次回、相手の石が1個追加されていれば履歴に追記します。石の削除・置換や複数の差分があれば、入力から履歴を組み直します。

> 初期化・盤面再構成・初期評価は探索時計の開始前に行われます。そのため、関数全体の所要時間は AI_THINK_TIME_MS と一致しません。

04 対局規則と禁じ手

白と黒で制限が異なります。規則はコンパイル時の設定に従います。

対象	既定の扱い
白	盤内の空点なら合法。三三・四四・長連の禁止は適用しない。
黒の三三	AI_ENABLE_DOUBLE_THREE=1: 禁じ手として除外。
黒の四四	AI_ENABLE_DOUBLE_FOUR=0: 既定では禁じ手として除外しない。
黒の長連	AI_ENABLE_OVERLINE=1: 6個以上の連続を除外。
黒の五連	長連でなければ、ちょうど5個の成立時は三三・四四検査を免除。

石の形の例



活三　両端が空いた三。四に発展する脅威。



活四　両端が空いた四。五連完成点が2か所。



長連　6個以上の連続。黒では既定で禁じ手。

上図は用語を理解するための代表例です。実装は、石が離れた形、複数方向の組合せ、補助的な形も判定します。三三は三の脅威が複数生じる形、四四は四の脅威が複数生じる形です。

黒の判定手順

① 盤内/空点と判定キャッシュを確認。② 仮着手点を含む縦・横・斜めの4方向の連続数を数える。③ 長連なら禁止。④ 五連成立、または三三/四四が両方無効なら許可。⑤ 周囲±6に黒石が3個未満なら許可。⑥ それ以外は着手前後の形状変化を調べる。

libraryForbiddenChange() は、仮着手の影響を受ける黒石について、7点の線形状、三/四の方向、飛び四の増加を比較します。以前からある脅威を新しい禁じ手に数えないため、変化前後を分けて扱います。

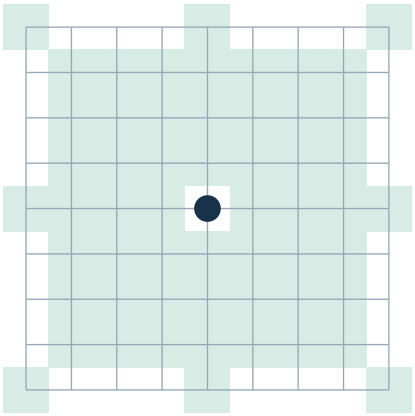
これは添付コードの実装仕様です。公式ルールへの完全準拠を認証したものではありません。対局側と同じ設定にしてください。既定では四四を禁止しない点が特に重要です。

05 候補手の生成と優先順位

全空点を毎回同じようには読みません。緊急の勝ち・防御を先に生成します。

優先	根の局面の条件	作る候補
1	自分に五連完成点がある	五連を作る点。
2	相手に五連完成点がある	その完成点を塞ぐ点。
3	自分に活四等を作る点がある	その必勝脅威を作る点。
4	相手に活四等を作る点がある	防御点を復元・選別し、自分の四の反撃も追加。
5	相手に有効な四三がある	四三への防御。防御があれば反撃追加、なければ通常候補。
6	上記のどれでもない	近傍の通常候補。

「五連完成点」は、まだ空いている点に置くと五連になる場所です。「活四等」は内部の OPEN_FOUR 分類で、活四のほか複数の四による複合脅威も含みます。



中央の黒点は既存の石。
緑色は通常候補を登録する近傍。

既定値3は「縦横±3の正方形」
と「縦・横・斜めの距離4」
を組み合わせる。

複数の石があれば範囲の和集合を使う。
盤外・占有点・黒の禁じ手は除外する。

根では合法手を選別し、候補がゼロなら通常候補を再生成して再選別します。その後、盤面を保つ8種類の回転・反転を調べ、等価な手をまとめます。盤端から始まる序盤では候補範囲を追加で広げます。

探索中は、置換表の有力手を先に試し、必要になってから他の候補を生成します。方策・履歴の得点で探索順を調整します。根の候補生成だけで、最善手が確定するわけではありません。

06 先読みの方式と終了条件

基本は「自分の手 → 相手の応手」を交互に読み、最善の評価を選ぶ方式です。

反復深化とαβ探索

反復深化は、浅い探索を終えた後に深さを1ずつ増やす方法です。既定では深さ2から最大99まで試します。ただし、時間・訪問数・必勝判断などで先に終了します。最大99は毎手99手先まで読むことを意味しません。

αβ探索は、すでに比較結果を変えないと判断できる枝を省略します。値は現在の手番の立場で扱い、相手番へ移ると符号を反転します。αは改善したい下限、βは残りの枝を省略できる境界です。

機構	分かりやすい説明
PVS / LMR	候補を狭い窓で試す。低順位の候補は浅く読み、有望なら深く/広く読み直す。
探索延長	強制防御、特異な有力手、四の応酬などの続きを重点的に読む。
Aspiration	深さ5以上では前回評価±16から探索し、範囲を外れた側の境界を広げる。
連続四探索 (VCF)	通常の深さが尽きても、四を打つ攻撃と五連完成点への防御を交互に確認する。
置換表と枝刈り	既存結果の再利用、静的評価、空手の試し読み、低い方策などで探索を減らす。

いつ探索を終えるか

最大深さに達したとき、中断フラグが立ったとき、必勝/必敗の初検出から既定6反復を追加したとき、唯一の有効候補の初検出から既定4反復を追加したとき、または一反復の終了時に時間配分の目標を超えたときに終了します。

時間制限の基準は CPU時間です。既定では訪問1024回ごとに9970 ms (10000-30) を確認し、反復完了時はその90%に相当する8973 msを確認します。 ノード上限は既定0で無効です。

> 途中で止まった反復の未確定値はそのまま通常評価として扱いません。ただし返却する着手は現在の候補配列の先頭であり、「最後に完全終了した深さの候補を丸ごと復元する」処理はありません。

07 局面の評価とNNUE

「盤面の良さ」と「先に読む手」は、目的が異なる2種類の評価です。

比較	Value: 局面の評価	Policy: 候補の優先度
役割	勝敗の見込みを探索用の数値にする。	各候補を読む順序や削減判断に使う。
主な関数	rpEvaluate() / nnueValueLogits()	rpScoreMoves() / nnuePolicy()
出力	現在の手番に有利なら正、相手に有利なら負。	候補ごとの得点。最終着手座標そのものではない。
推論しない場合	古典的な形状評価を返す。	形状の得点を候補順の基礎にする。

Valueの処理

現在と直前の黒の形状評価を平均し、白番なら符号を反転して±6000に制限します。NNUEが有効で、古典評価が探索窓の余裕幅内にあるときだけ、NNUE推論へ進みます。常にニューラル推論するわけではありません。

NNUEは全盤と地域の特徴から出力を計算します。最初の3出力を勝・負・引分けとしてsoftmaxで正規化し、設定に応じて引分けの扱いを補正します。その後、勝敗の差から得た見込みを対数で評価値に変換し、±6000に収めます。

Policyの処理

候補マスクで必要な点だけを推論します。Policyが利用できるときは、その得点を基礎得点に用い、形状得点の基礎を置き換えます。履歴得点と、条件に応じた応手ボーナスを加えて部分整列します。

学習モデルと差分同期

黒用・白用モデルは文字列として埋め込まれています。初回に復号・展開・形式検査を行い、特徴辞書と各層を構築します。推論時は nnueSync() が探索経路の共通部分を保持し、分岐した部分を復元・差分更新します。実行中に学習する機能はありません。

評価領域	定数・意味
通常の静的評価	±6000。必勝値と区別する。
必勝・必敗	基準±30000。到達手数で補正する。絶対値29500以上を必勝領域として扱う。
番兵値	探索窓の外側=30001 / 未評価・中断=-30002。通常評価とは区別する。

08 内部状態とデータの寿命

高速化のため、盤面・形状・特徴を保持し、一手で変わる部分を更新します。

データ	役割	寿命
Position	AI専用盤面、空点形状、履歴状態、復元領域、候補ビット、黒合法性キャッシュ。	一回の関数呼出しで確保・解放。
RpState	手番、占有数、形状数、評価、候補境界、局面キー、直近の脅威位置。	Position内で手数ごとに保持。
RpStack	節点の評価、候補数、除外手、キラー手、主変化など。	一回の呼出しで確保・解放。
RpMovePicker / RpRootMove	候補を遅延生成する状態 / 根の候補の評価と主変化。	探索中 / 根の配列は静的領域。
置換表 (TT)	局面のキー断片、深さ、評価境界、有力手、世代。	呼出しをまたいで保持。必要時に初期化。
NNUEモデル・評価状態	黒/白の展開済み重み、特徴、評価済み盤面と探索経路。	初期化後は保持。解放用公開関数はない。
対局履歴・予想盤面	前回返した手と次回入力の差分を判断する。	呼出しをまたいで保持。

仮着手と取消し

rpMake() は親の状態を保存し、手番、盤面キー、占有数、形状、候補を更新します。rpUndo() は履歴を戻し、石と候補参照数を戻し、保存された形状を復元します。どちらも通常着手では黒の合法性キャッシュを無効化します。

空手 (move=-1) は、石を置かずに手番と局面キーを切り替える、内部探索だけの試し読みです。対局側にパスを返す機能ではありません。

盤面の内部座標

内部は幅32の余白付き盤面です。実座標 (x,y) は ((y+5)<<5)|(x+5) に変換し、周囲を壁として扱います。外部へは rpX()/rpY() で0始まりの座標に戻します。

> 複数の静的変数を共有する実装です。同時呼出しや複数対局の並列実行を安全に扱うロック・独立コンテキストはありません。利用側で直列化するか、別プロセスに分けてください。

09 主要な設定と調整方法

設定は #ifndef 付きマクロです。既定値をソース編集や -D 指定で変更できます。

設定名	既定	意味・変更時の影響
AI_THINK_TIME_MS	10000.0	探索のCPU時間予算。増やすと読める量が増えるが、全体の応答時間保証ではない。
AI_START_DEPTH / AI_MAX_SEARCH_DEPTH	2 / 99	反復深化の開始/上限。実際の深さは削減・延長で変わる。
AI_MAX_SEARCH_PLY	256	通常探索・延長・VCFを含む探索手数の保存上限。
AI_NODE_LIMIT	0	訪問数の上限。0は無効。正ならその回数に到達すると停止。
AI_TIME_CHECK_INTERVAL	1024	制限確認の訪問間隔。小さくすると頻繁に確認する。
AI_CANDIDATE_RANGE	3	通常候補の近傍範囲。5は全盤。広くすると候補と計算量が増える。
AI_TRANSPOSITION_TABLE_BITS	18	TTの区画数の指数。1増やすごとに容量が2倍。
AI_PRINCIPAL_VARIATION_COUNT	1	上位候補を個別に探索する数。公開関数が返すのは1手。
AI_ENABLE_RAPFI / NNUE_VALUE / NNUE_POLICY	すべて1	後2つの正式名は AI_ENABLE_NNUE_VALUE / AI_ENABLE_NNUE_POLICY。モデル・局面推論・方策を選択。

変更する順序

まず対局側と禁じ手の設定を一致させます。次に許容できる応答時間を決め、時間予算と候補範囲を調整します。TT容量は使用環境のメモリに合わせます。細かな探索係数は、有効な局面群で結果を比較しながら変更します。

例: 5秒の探索予算にする指定

```
gcc -std=c11 -O3 -DAI_THINK_TIME_MS=5000.0 ... -lm
```

値	AI_CANDIDATE_RANGE の意味
0 / 1	縦横±2の正方形 / 同正方形+直線・斜めの距離3
2 / 3	縦横±3の正方形 / 同正方形+直線・斜めの距離4
4 / 5	縦横±4の正方形 / 全盤

機能切替・探索延長・削減・履歴・評価など、変更可能な119設定の既定値とコメントを付録Aに掲載します。付録はソースから抽出した現行値です。

10 ビルド条件・時間・メモリ

必要な定義と資源を先にそろえてから、対局プログラムに組み込みます。

依存とコンパイル

必要なローカルヘッダーは gomoku.h です。BOARD_SIZE、STONE_SPACE/BLACK/WHITE、LEFT/TATE/YOKO/RIGT、SANSAN/SHISHI/CHOLEN などの定義と公開関数宣言が必要です。添付されていないため、これらの実定義は本書では確認できません。

標準ライブラリのメモリ確保、時計、数学関数を使用します。GCC/Clangでは _Alignas を含むC11対応が必要です。Linuxでの組込み例は次のとおりです。実際のリンク対象は対局プログラムに合わせてください。

```
gcc -std=c11 -Wall -O3 -c gomoku_ai.c -o gomoku_ai.o
gcc <game_objects> gomoku_ai.o -lm -o gomoku
```

SSE2対応環境ではNNUEの整数演算にSSE2を選択します。非対応環境や NNUE_FORCE_SCALAR=1 では標準Cの演算経路を使います。既定のNNUE初期化は BOARD_SIZE==15 を要求します。

主なメモリ使用

領域	容量・扱い
置換表	64バイト×2^18区画=16 MiB。各区画5件で最大1,310,720件。1件12バイト。
NNUE特徴辞書	黒/白それぞれ 2×65536×64×sizeof(int16_t)=16 MiB。合計32 MiB。
埋込みデータ・展開領域	形状/防御表、モデル文字列、展開済みモデルを保持。ソース容量自体は約30 MB。
一手ごとの領域	sizeof(Position) と (SEARCH_PLY_LIMIT+7)×sizeof(RpStack) を確保。
NNUE復元経路	黒/白で (SEARCH_PLY_LIMIT+1)×sizeof(NnueRestore) を必要時に確保。確保失敗時は盤面比較の同期へ戻る。

コンパイル時に確認する範囲

禁じ手設定は0/1。TT指数は1 ~ 24。探索手数上限は8 ~ 256。開始/最大深さは1 ~ 200。時間確認間隔は1以上。候補範囲は0 ~ 5。PV数は1 ~ 盤点数です。時間予算は整数化した値が正であることを要求します。

> TTだけを見て必要メモリを判断しないでください。モデル・特徴辞書・復元領域も必要です。総使用量とピーク使用量、関数の実所要時間は実行環境で測定する必要があります。

11 失敗時の動作と利用上の制約

戻り値1は、AIが着手座標を正常に返せなかったことを表します。

原因	動作と利用側の対応
不正な引数・盤面値	失敗を返す。盤面サイズ、石色、出力先、各マスの値を修正する。
NNUE初期化失敗	モデル機能が有効なら失敗。15×15、モデル形式、メモリを確認する。初期化失敗は状態に保持され、自動再試行しない。
Position/スタック確保失敗	確保済み領域も解放して失敗。使用可能メモリと設定を確認する。
満盤	公開関数は失敗1。探索内部の満盤評価0と区別する。
候補が得られない	失敗。候補範囲・合法性選別・対称性除去後の結果であり、盤面全体の合法手不存在を常に証明したものではない。
時間/訪問数による中断	通常は現在の先頭候補を返して成功。エラーコードや到達深さは公開しない。

呼び出し側が保証すること

有効なメモリ領域と整合する石の定数を渡します。対局が未終了であること、実際にAIの手番であること、黒白の石数や着手履歴が対局ルールに整合することは、呼び出し側で確認してください。入力検査は盤面値を確認しますが、それらの整合性は検証しません。

公開していない機能

評価値、探索深さ、訪問数、複数候補、主変化は戻り値として公開しません。学習、GUI、ネットワーク通信、対局保存、明示的な対局リセット、常駐領域の終了時解放の公開APIもありません。
AI_ENABLE_SEARCH_TRACE を定義した検証用構成では、標準エラーに探索ログを出す処理があります。

性能と再現性

乱数によるランダム着手は行わず、固定種から局面キーを構築します。ただし探索の停止時刻、実行速度、設定、同一プロセスの置換表履歴などにより、選ぶ手は変わり得ます。先読みは候補制限と枝刈りを使うため、全合法手を最後まで読み切った最善性や勝利を保証するものではありません。

> NNUEモデル読込みの失敗時に、自動で古典評価へ切り替えて成功する仕様ではありません。古典評価だけで使う場合は、コンパイル時にモデル関連の設定を変更します。

12 主要関数と確認項目

保守するときは、責務ごとの関数群を入口にすると追いやすくなります。

責務	主な関数
公開窓口	calcPutPos
初期化・盤面入力	rpInitialize / nnueInitialize / rpLoadBoard / rpExpandOpeningCandidates
合法性と禁じ手	isLegalMove / rpLegal / isForbiddenBlackMove / libraryForbiddenChange / libraryShapeAt
形状・差分更新	rpAggregate / rpUpdateLineCell / rpMake / rpUndo / rpClear
候補・防御	rpGenerateRoot / rpInitPicker / rpFillPicker / rpNextMove / rpFourDefence / rpB4F3Defence / rpFilterSymmetry
先読み・早期勝敗	rpSearch / rpAspiration / rpVCFSearch / rpVCFDefend / rpQuickWin
局面評価・方策	rpEvaluate / rpScoreMoves / nnueValueLogits / nnuePolicy / nnueSync
結果再利用・制限	rpProbe / rpStore / rpSortRoot / rpUpdateHistory / rpTouchNode

対局側に組み込んだ後の確認例

確認対象	期待する動作
空盤・満盤	15×15空盤なら(7,7)と成功0。満盤なら失敗1。
入力異常	NULL、同一の出力先、不正石色、不正盤面値で失敗1。
即勝ちと単一防御	自分の合法な五連完成点を選ぶ。相手の唯一の五連完成点を塞ぐ。
禁じ手設定	黒の長連・三三を候補から除く。五連優先と四四の切替を確認。
盤面と座標	入力盤面が変わらず、成功時の出力が盤内の合法な空点である。
継続対局と中断	相手の1手追加、盤面組直し、短い時間予算でも状態が整合する。

上表は実施すべき確認例であり、本書作成時に実行した試験の結果ではありません。対局側の規則に合わせて、複数方向の三/四と盤端の局面も確認します。

用語: 節点=探索中の局面 / root=現在の入力局面 / ply=片側の1着手 / PV=選んだ手の列 / TT=置換表 / NNUE=特徴を差分更新できるニューラル評価 / VCF=連続四による強制攻撃。

付録A 設定値の完全一覧 (1/7)

以下は #ifndef で外部指定を許す119設定です。数値と説明は添付ソースから抽出しています。0/1の機能設定は0が無効、1が有効です。内部の固定定数はこの一覧に含めません。

設定名	既定値	役割 (ソースコメント)
AI_ENABLE_DOUBLE_THREE	1	対局規則に合わせて黒の三三を除外するための切替。 1 で有効。
AI_ENABLE_DOUBLE_FOUR	0	対局規則に合わせて黒の四四を除外するための切替。 1 で有効。
AI_ENABLE_OVERLINE	1	対局規則に合わせて黒の長連を除外するための切替。 1 で有効。
AI_THINK_TIME_MS	10000.0	応答時間を制限する一手当たりのミリ秒。初回の重み展開は含めない。
AI_MAX_SEARCH_DEPTH	99	反復深化の計算量を制限する上限深さ。
AI_START_DEPTH	2	初回から一定の読みを確保する反復深化の開始深さ。
AI_MAX_SEARCH_PLY	256	延長と静止探索を含めても保存領域を越えないための手数上限。
AI_TRANSPOSITION_TABLE_BITS	18	再利用できる局面数とメモリ量を調整する区画数の指数。区画数は 2 のべき乗。
AI_NODE_LIMIT	0	時間とは別に計算量を制限する訪問数。 0 なら訪問数では中断しない。
AI_TIME_CHECK_INTERVAL	1024	中断の応答性と時計参照の負荷を調整する訪問間隔。
AI_TIME_RESERVED_MS	30.0	時間確認の遅れを吸収するために確保する余裕のミリ秒。
AI_TIME_OPTIMUM_RATIO	0.9	次の反復で上限を越えにくくするための時間配分比率。
AI_CANDIDATE_RANGE	3	近傍の読みと候補数を調整する範囲。 0 = 2 点四角、 1 = 2 点四角と 3 点直線、 2 = 3 点四角、 3 = 3 点四角と 4 点直線、 4 = 4 点四角、 5 = 全盤。
AI_ENABLE_ROOT_SYMMETRY	1	等価な根の候補へ重複して時間を使わないための対称性除去。
AI_ENABLE_RAPFI	1	公開モデルを組み込むための切替。 0 なら古典評価だけで動作する。
AI_ENABLE_NNUE_VALUE	1	学習した勝敗の見込みを局面比較に使うための切替。
AI_ENABLE_NNUE_POLICY	1	有力な着手へ先に探索時間を使うための学習方策の切替。

付録A 設定値の完全一覧 (2/7)

項目 18 ~ 34 / 119。設定はソース内の記述順に掲載。

設定名	既定値	役割 (ソースコメント)
NNUE_FORCE_SCALAR	0	整数演算の一致を比較できるように標準 C の経路を強制する切替。
AI_ENABLE_POLICY_MASK	1	不要な空点の方策計算を省くための候補マスクの切替。
AI_ENABLE_NNUE_STATE_CACHE	1	兄弟の枝へ戻る際の特徴再計算を省くための保存機能の切替。
AI_ENABLE_BOARD_RESTORE	1	着手取消し時の形状表引きを省くための復元機能の切替。
AI_ENABLE_CANDIDATE_BITS	1	候補でない点の走査を省くためのビット集合の切替。
AI_ENABLE_TRANSPOSITION_PREFETCH	1	子局面でのメモリ待ちを減らすための置換表先読みの切替。
AI_PRINCIPAL_VARIATION_COUNT	1	上位候補を個別に比較するための探索数。公開関数はその最上位だけを返す。
AI_ENABLE_ASPIRATION	1	前回評価付近の狭い窓で探索量を減らすための切替。
AI_ENABLE_RAZORING	1	浅い局面の見込みを連続四で確認して通常探索を省くための切替。
AI_ENABLE_FUTILITY	1	評価差を覆しにくい枝へ時間を使いすぎないための切替。
AI_ENABLE_NULL_MOVE	1	手番を譲っても有利な局面の探索を省くための試し読みの切替。
AI_ENABLE_INTERNAL_ITERATIVE_REDUCTION	1	参考手の確かさに応じて内部節点の深さを減らすための切替。
AI_ENABLE_INTERNAL_ITERATIVE_DEEPENING	1	参考手のない深い節点で先に探索順を得るための切替。
AI_ENABLE_LATE_MOVE_REDUCTION	1	順位の低い候補をまず浅く読み、探索量を抑えるための切替。
AI_ENABLE_SINGULAR_EXTENSION	1	唯一有力な手の続きを重点的に読むための切替。
AI_ENABLE_POLICY_PRUNING	1	学習方策で見込みの薄い候補を省くための切替。
AI_ASPIRATION_DEPTH	5.0f	前回評価が安定してから狭い窓を使うための開始深さ。

付録A 設定値の完全一覧 (3/7)

項目 35 ~ 51 / 119。設定はソース内の記述順に掲載。

設定名	既定値	役割 (ソースコメント)
AI_ASPIRATION_DELTA	16	前回評価の変動を許容する初期窓の半幅。
AI_INTERNAL_DEEPENING_MIN_DEPTH	14.7f	事前探索の負荷が見合う局面に限定する最小深さ。
AI_INTERNAL_REDUCTION_BASE	0.69f	参考手を得る事前探索の負荷を抑える本探索の深さ減少量。
AI_INTERNAL_REDUCTION_PRINCIPAL_VARIATION	0.32f	参考手のない主変化節点の探索量を抑える深さ減少量。
AI_INTERNAL_REDUCTION_TABLE_SCALE	0.28f	置換表との深さ差から削減量を求めるための倍率。
AI_INTERNAL_REDUCTION_TABLE_MAX	3.6f	置換表の深さが浅くても削減しすぎないための上限。
AI_TRIVIAL_MOVE_PRUNING_DEPTH	4.3f	重要度の低い候補を深い節点で省くための深さ基準。
AI_ALPHA_IMPROVEMENT_REDUCTION	1.0f	最善評価が更新された後の候補へ使う時間を抑える深さ減少量。
AI_OPPONENT_FIVE_EXTENSION	1.3f	強制防御で読みの先端が止まらないようにする深さ延長量。
AI_SINGULAR_EXTENSION_MIN_DEPTH	6.65f	特異性の試し読みを十分深い節点に限定する最小深さ。
AI_SINGULAR_EXTENSION_TABLE_DEPTH_MARGIN	2.03f	参考手を信頼できる置換表の深さ差の許容幅。
AI_SINGULAR_EXTENSION_EXTRA_DEPTH_LIMIT	10.8f	特異延長の繰返しで探索が伸びすぎないための累積上限。
AI_SINGULAR_EXTENSION_FAIL_HIGH_REDUCTION	1.5f	特異性の検証で上限を超えた節点の探索量を抑える削減量。
AI_SINGULAR_EXTENSION_VALUE_MARGIN_SCALE	0.94f	他の候補が劣ると判断する評価差を深さに応じて広げる倍率。
AI_SINGULAR_EXTENSION_SEARCH_REDUCTION_SCALE	0.54f	他候補の比較に使う検証探索を浅くするための倍率。
AI_TABLE_MOVE_PRINCIPAL_VARIATION_EXTENSION	0.25f	主変化で参考手の続きを重点的に読むための延長量。
AI_TABLE_MOVE_NON_PRINCIPAL_VARIATION_EXTENSION	0.08f	主変化以外でも参考手の読みを補うための延長量。

付録A 設定値の完全一覧 (4/7)

項目 52 ~ 68 / 119。設定はソース内の記述順に掲載。

設定名	既定値	役割 (ソースコメント)
AI_NEARBY_FOUR_EXTENSION_DISTANCE_FOUR	0.24f	直近の攻撃に近い四の応酬を読み切るための延長量。距離 4 以内。
AI_NEARBY_FOUR_EXTENSION_DISTANCE_SIX	0.05f	少し離れた四の応酬も読みを補うための延長量。距離 6 以内。
AI_CONTINUOUS_ATTACK_EXT	0.53f	連続した攻撃の途中で探索が終わらないようにする延長量。
AI_LATE_MOVE_REDUCTION_LOG_SCALE	0.75f	深さと候補順位の対数から削減量を求める倍率。
AI_LATE_MOVE_REDUCTION_EXTRA_MAX_DEPTH	5.0f	状況別の追加削減が深くなりすぎないための深さ基準。
AI_TABLE_PRINCIPAL_VARIATION_REDUCTION_DISCOUNT	0.97f	置換表の主変化を見落としにくくする削減量の割引。
AI_NO_ALPHA_IMPROVING_REDUCTION	0.98f	最善評価を更新しない枝に追加する深さ削減量。
AI_CUT_NODE_WITHOUT_KILLER_REDUCTION	1.8f	有力なキラー手のないカット節点の計算量を抑える削減量。
AI_DISTANT_OPPONENT_DEFENCE_REDUCTION	1.93f	相手の直近攻撃から遠い防御候補に使う時間を抑える削減量。
AI_DISTANT_SELF_DEFENCE_REDUCTION	0.96f	自分の直近の四から遠い候補に使う時間を抑える削減量。
AI_PSEUDO_FORBIDDEN_REDUCTION_DISCOUNT	1.05f	見かけ上の禁じ手形状を過度に削減しないための割引。
AI_COMPLEXITY_TRIVIAL_DISTRACT	0.063f	低重要度の離れた手の削減を局面の複雑さに合わせる倍率。
AI_COMPLEXITY_TRIVIAL	0.028f	低重要度の手の削減を局面の複雑さに合わせる倍率。
AI_COMPLEXITY_UNIMPORTANT	0.023f	補助的な手の削減を局面の複雑さに合わせる倍率。
AI_COMPLEXITY_IMPORTANT	0.007f	有力な手の削減を局面の複雑さに合わせる倍率。
AI_POLICY_REDUCTION_BASE	4.0f	低い方策得点の候補を浅く読むための削減基準。
AI_POLICY_REDUCTION_SCALE	3.2f	方策得点から深さ削減量へ換算する倍率。

付録A 設定値の完全一覧 (5/7)

項目 69 ~ 85 / 119。設定はソース内の記述順に掲載。

設定名	既定値	役割 (ソースコメント)
AI_POLICY_REDUCTION_MAX	5.1f	方策が低くても読みを削りすぎないための上限。
AI_POLICY_PRUNE_BASE	376	方策で候補を省く判断に余裕を持たせる得点基準。
AI_POLICY_PRUNE_SCALE	63.0f	深い節点ほど方策による枝刈り条件を調整する倍率。
AI_RAZOR_DEPTH	2.6f	連続四だけで低評価を確認する処理を浅い節点に限定する深さ。
AI_RAZOR_QUADRATIC	0.17f	低評価とみなす余裕を深さの二乗に応じて広げる係数。
AI_RAZOR_LINEAR	14.0f	低評価とみなす余裕を深さに応じて広げる係数。
AI_RAZOR_CONSTANT	19	低評価による枝刈りに最低限の余裕を持たせる評価幅。
AI_FUTILITY_SCALE	80	探索で評価が改善する可能性を深さに応じて見積もる評価幅。
AI_FUTILITY_NO_TT_SCALE	12	参考情報のないカット節点の無益性判断を調整する評価幅。
AI_FUTILITY_COUNT_BASE	3	候補数による枝刈りでも最低限の手を読むための基準数。
AI_FUTILITY_COUNT_EXPONENT	1.32	深い節点で読む候補数を増やすための指数。
AI_NULL_MIN_DEPTH	8.0f	空手の試し読みが見合う深い節点に限定する開始深さ。
AI_NULL_MARGIN_BASE	624	空手を試す前に優位が十分あることを求める評価差。
AI_NULL_MARGIN_SCALE	28	空手を試す優位の基準を深さに合わせる係数。
AI_NULL_MARGIN_MAX	230	空手の評価差基準の補正が大きくなりすぎないための上限。
AI_NULL_REDUCTION_BASE	4.29f	手番を譲る試し読みに使う計算量を抑える深さ削減の基準。
AI_NULL_REDUCTION_SCALE	0.17f	空手の試し読みの削減量を深さに合わせる係数。

付録A 設定値の完全一覧 (6/7)

項目 86 ~ 102 / 119。設定はソース内の記述順に掲載。

設定名	既定値	役割 (ソースコメント)
AI_INTERNAL_DEEPENING_REDUCTION_BASE	10.0f	参考手を得る事前探索を本探索より浅くするための基準。
AI_INTERNAL_DEEPENING_REDUCTION_SCALE	0.57f	事前探索の深さ削減を本探索の深さに合わせる係数。
AI_FAIL_HIGH_BASE	-9	上限超過の見込みを判定して候補の扱いを変えるための評価差基準。
AI_FAIL_HIGH_SCALE	15	上限超過の見込みを判定する余裕を深さに合わせる係数。
AI_FAIL_HIGH_FOUR	87	強制的な脅威がある局面の上限超過判断を慎重にする評価幅。
AI_CONTINUOUS_FOUR_SEARCH_MARGIN_BASE	2158	連続四で静的評価からの改善を見込む基準幅。
AI_CONTINUOUS_FOUR_SEARCH_MARGIN_SCALE	57	連続四の探索を省く評価幅を深さに合わせる係数。
AI_CONTINUOUS_FOUR_SEARCH_MARGIN_MIN	768	連続四による逆転を見落としにくくする評価幅の下限。
AI_HISTORY_SCORE_RANGE	10692	履歴得点が際限なく増え、探索順を独占しないための範囲。
AI_HISTORY_BONUS_QUADRATIC	25.0f	深い探索で有効だった手を優先する履歴加点の二次係数。
AI_HISTORY_BONUS_LINEAR	105.0f	探索深さを履歴の加点量へ換算する一次係数。
AI_HISTORY_BONUS_CONSTANT	-157.0f	浅い探索の履歴加点を抑えるための定数項。
AI_HISTORY_BONUS_MAX	8927	一回の探索で履歴得点を変えすぎないための加点上限。
AI_STAT_QUIET_WEIGHT	780	静かな手の実績を深さ削減へ反映するための倍率。
AI_STAT_OFFSET	3322	履歴得点による深さ削減の基準位置を調整する補正値。
AI_STAT_DIVISOR	12267	履歴得点を通常の削減量へ換算する尺度。
AI_STAT_DEEP_DIVISOR	4380	履歴得点を深い再探索の削減量へ換算する尺度。

付録A 設定値の完全一覧 (7/7)

項目 103 ~ 119 / 119。設定はソース内の記述順に掲載。

設定名	既定値	役割 (ソースコメント)
AI_COUNTER_MOVE_BONUS	21	直前の着手に有効だった応手を先に読むための加点。
AI_LATE_MOVE_REDUCTION_DEEPER_BASE	34	浅い探索で有望だった手を深く読み直すための評価差基準。
AI_LATE_MOVE_REDUCTION_DEEPER_SCALE	11.0f	深い再探索が必要な評価差を深さに合わせる係数。
AI_LATE_MOVE_REDUCTION_EVEN_DEEPER_BASE	288	さらに有望な手に追加の読みを与えるための評価差基準。
AI_LATE_MOVE_REDUCTION_EVEN_DEEPER_SCALE	26.0f	追加の再探索が必要な評価差を深さに合わせる係数。
AI_DOUBLE_SINGULAR_EXTENSION_MARGIN_BASE	29	二重の特異延長に十分な他候補との評価差を求める基準。
AI_DOUBLE_SINGULAR_EXTENSION_MARGIN_SCALE	0.6f	二重の特異延長の評価差基準を深さに合わせる係数。
AI_DOUBLE_SINGULAR_EXTENSION_MARGIN_MAX	9	二重の特異延長の評価差補正に設ける上限。
AI_ITERATIONS_AFTER_MATE	6	必勝が見えても追加確認するための反復回数。
AI_ITERATIONS_AFTER_SINGULAR	4	有効な根の手が一つでも追加確認するための反復回数。
AI_EVAL_SCALING_FACTOR	200.0f	公開モデルの勝敗確率を探索の評価尺度へ合わせる倍率。
AI_EVAL_MARGIN_SCALE	395.0f	評価窓から必要な勝敗確率の精度を求める尺度。
AI_EVAL_MARGIN_WINLOSS_SCALE	1.18f	勝敗差による評価余裕を確率へ換算する倍率。
AI_EVAL_MARGIN_EXPONENT	3.07f	確率に応じて評価の許容幅を変えるための指数。
AI_DRAW_RATIO	1.0f	引分け確率を探索用の勝率へ換算するための重み。
AI_DRAW_BLACK_WIN_RATE	0.5f	引分けを黒側の期待値へ配分するための比率。
AI_POLICY_SCORE_SCALE	32.0f	方策出力を探索順の整数得点へ合わせる倍率。